1 Para qué sirve esta guía

> *Para entender el proyecto parte por parte y poder comprobar por tu cuenta si cada patrón está bien aplicado, con el código a la vista.*

La guía recorre el Sokoban tal como está hoy en el repositorio: qué hace cada clase, cómo se conectan y dónde está cada patrón. Cada patrón trae una lista de chequeo con el archivo y la línea donde mirar, así no hace falta creerle a nadie: abrís el archivo y lo comprobás.

La consigna deja claro por qué esto importa: **«El foco de la evaluación es la correcta aplicación de los principios y patrones de diseño presentados en la asignatura (SOLID, GRASP, MVC, GOF)»**, y además **«Cada integrante del equipo deberá realizar una defensa exhaustiva del proyecto (oral y/o escrita), lo que determinará la calificación final del mismo.»**

15	3 + 1	28 / 101	2
clases, interfaces, enums y records (14 archivos .java + el record interno Foto)	patrones GoF (Observer, Facade, Singleton) + arquitectura MVC	imágenes de Martín que ya usa el juego	partes pendientes: dominio/personaje y música/sonido

LA IDEA FUERZA

La vista (Luca) y el dominio (pendiente) no se conocen: los dos dependen de un contrato de interfaces (`Juego` , `ObservadorJuego` , `EstadoJuego`). Por eso cada uno puede trabajar por separado y el dominio se enchufa cambiando una sola línea en `Main` .

1.1 Cómo leer este apunte

Color	Qué marca
Flúor y negrita	Texto literal de la consigna del profesor (Prof. Pepe). Es lo más importante.
Naranja	Rótulos y conceptos que ordenan el texto.
Amarillo	Definiciones.
Celeste	Explicación técnica: cómo y por qué funciona.
Verde	Lo que está bien hecho, buenas prácticas, ventajas.
Rojo	Advertencias, riesgos, cosas a mejorar.
Violeta	Ampliación tomada de la bibliografía (GoF), no de lo que hay en el proyecto.
Cian	Rótulo de ejercicio resuelto.

En las listas de chequeo: **✓ se cumple** **⚠ a medias / revisar** **× no se cumple todavía**. Las referencias tipo `vista/Imagenes.java:29` son archivo y número de línea dentro de `src/main/java/sokoban/` . En los recuadros de código, los comentarios en dorado se agregaron para esta guía y no están en el archivo original.

2 Cómo se lee un diagrama de clases

> *Lo mínimo para leer cualquier diagrama del proyecto sin perderse: la caja de cada clase, los símbolos y, sobre todo, hacia dónde apuntan las flechas.*

2.1 La caja de una clase

Tres compartimentos

Arriba el nombre; en el medio los atributos (lo que la clase guarda); abajo los métodos (lo que la clase hace). Si un compartimento está vacío igual se dibuja la rayita.

Visibilidad

`+` público (lo usa cualquiera), `-` privado (solo la clase), `#` protegido (la clase y sus hijas).

Subrayado y cursiva

Subrayado = `static`: pertenece a la clase y no a cada objeto (por ejemplo `obtenerInstancia()`).
Cursiva = abstracto: un método sin código, como todos los de una interfaz.

Estereotipos «...»

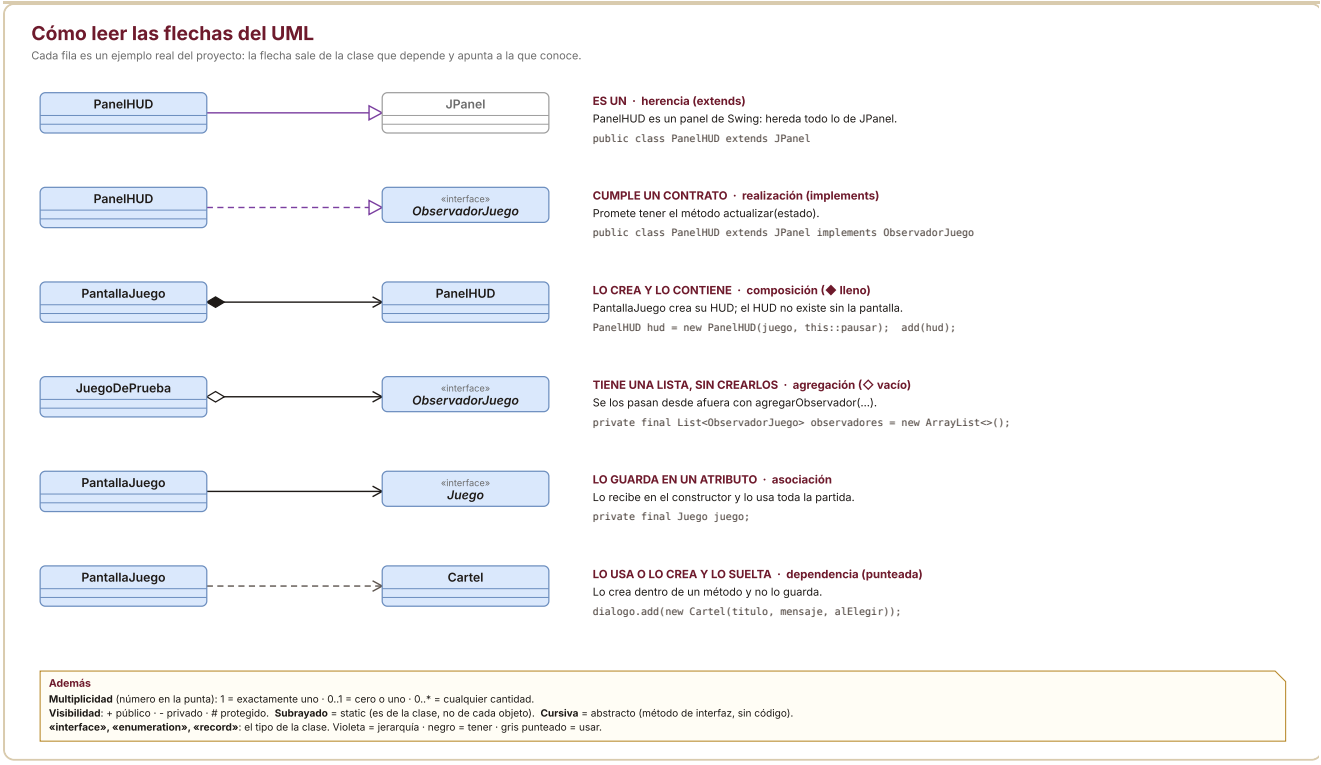
Dicen qué tipo de cosa es: «interface», «enumeration», «record». En los diagramas de patrones, la etiqueta dorada arriba de la caja dice el **rol** que cumple esa clase en el patrón.

Colores de las cajas

Autoría: azul = Luca, verde = Martín, amarillo = integración, gris punteado = pendiente.

2.2 Las flechas

Regla general: **la flecha sale de la clase que depende y apunta a la que conoce.** Si dudás, preguntate «¿quién tiene el código que menciona a quién?»: la flecha sale de ese archivo.



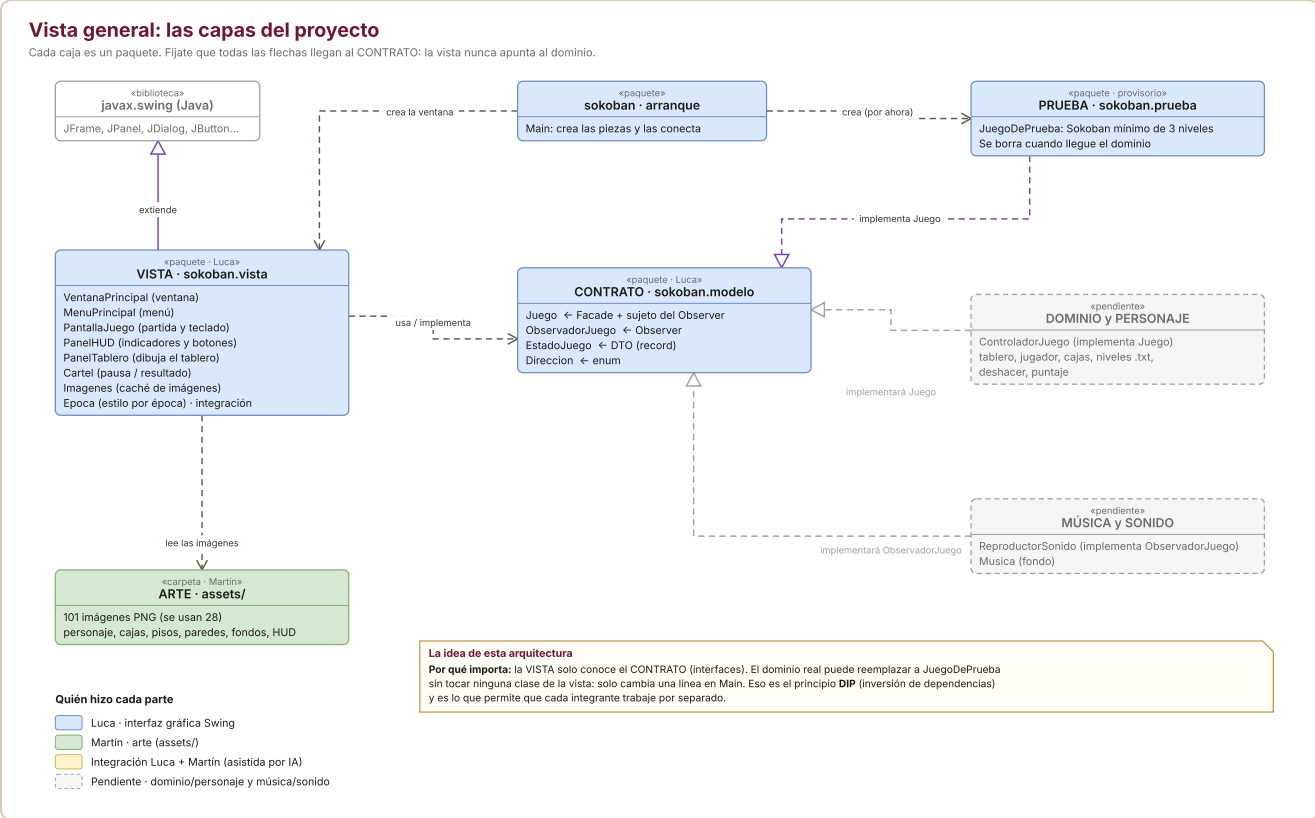
docs/uml/0-como-leer-flechas · cada fila es un ejemplo real del proyecto.

2.3 Errores típicos al leer o dibujar

- Dibujar la flecha al revés. `PantallaJuego` conoce a `Juego`, no al revés: la flecha va de `PantallaJuego` hacia `Juego`.
- Confundir herencia con realización. Herencia (`extends`) es línea llena; realización (`implements`, con una interfaz) es punteada. Las dos terminan en triángulo hueco.
- Confundir composición con agregación. Composición (◆ lleno): el todo crea a la parte y la parte no vive sin él. Agregación (◇ vacío): el todo tiene a la parte pero se la dieron de afuera.
- Poner asociación donde solo hay un uso pasajero. Si la otra clase aparece solo dentro de un método (se crea o se recibe y no se guarda), es dependencia (punteada), no asociación.

3 El mapa del proyecto

> *Antes de entrar en las clases, la foto desde lejos: qué paquetes hay, quién hizo cada uno y quién depende de quién.*



docs/uml/1-vista-general · todas las flechas llegan al CONTRATO.

Paquete	Qué contiene	Autor	Estado
`sokoban`	Main: arranca el programa y conecta las piezas.	Luca	✓ listo
`sokoban.vista`	Todo lo que se ve: ventana, menú, partida, HUD, tablero, carteles, imágenes.	Luca (+ `Epoca` en la integración)	✓ listo
`sokoban.modelo`	El contrato entre la vista y el dominio: 2 interfaces, 1 record y 1 enum.	Luca	✓ listo
`sokoban.prueba`	JuegoDePrueba: un Sokoban mínimo para probar la vista.	Luca	⚠ provisorio
`assets/`	101 imágenes: personaje, cajas, pisos, paredes, fondos, HUD.	Martín	✓ listo (28 en uso)
dominio y personaje	Tablero, jugador, cajas, niveles .txt, deshacer, puntaje.	—	× pendiente
música y sonido	Efectos por acción y música de fondo.	—	× pendiente

PARA VALIDAR

Ningún archivo de `sokoban.modelo` ni de `sokoban.prueba` importa algo de `sokoban.vista`. Si algún día aparece un `import sokoban.vista…` en el dominio, se rompió la arquitectura. Comprobalo buscando «import sokoban.vista» en esos paquetes: hoy no aparece ninguno.

4 Qué pasa cuando jugás

> *El diagrama de clases muestra la estructura; el de secuencia muestra el orden de las llamadas. Seguir una jugada de punta a punta es la mejor forma de entender el diseño.*

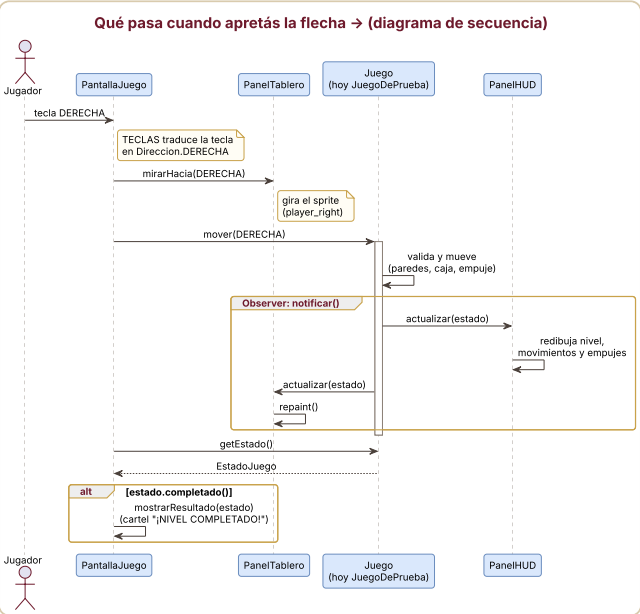
4.1 El arranque

Main.java

```
18  public static void main(String[] args) {
19      SwingUtilities.invokeLater(() -> {          // todo Swing corre en un solo hilo (EDT)
20          Juego juego = new JuegoDePrueba();      // ÚNICA línea que conoce la clase concreta
21          new VentanaPrincipal(juego).setVisible(true);
22      });
23  }
```

`Main` es el único lugar que dice `new JuegoDePrueba()`; el resto del programa recibe un `Juego` (la interfaz). Cuando llegue el dominio se cambia esta línea y nada más. `VentanaPrincipal` crea el menú y la pantalla de juego, y la pantalla de juego crea el HUD y el tablero y los registra como observadores.

4.2 Apretar una flecha, paso a paso



docs/uml/7-secuencia-mover

1. `PantallaJuego` recibe la tecla y la traduce con el mapa `TECLAS` en `Direccion.DERECHA` (`vista/PantallaJuego.java:56`).
2. Gira el personaje: `tablero.mirarHacia(DERECHA)` (usa el sprite `player_right` de Martín).
3. Le pide al juego `mover(DERECHA)`: es lo único que la vista sabe hacer con el dominio.
4. El juego valida el movimiento (paredes, cajas, empuje) y, si cambió algo, llama a `notificar()`.
5. Observer: `notificar()` recorre la lista y llama `actualizar(estado)` en el HUD y en el tablero; cada uno se redibuja solo.
6. `PantallaJuego` pide `getEstado()`; si `completado()` es verdadero, muestra el cartel de resultado (`vista/PantallaJuego.java:75`).

Fijate que en ningún paso la vista toca el tablero del dominio: solo pide y recibe una foto (`EstadoJuego`).

5 Clase por clase

> *Qué hace cada clase, con quién se relaciona y qué mirar para validarla. Está ordenado por paquete, de afuera (lo que se ve) hacia adentro (el contrato).*

5.1 Vista · `sokoban.vista` (Luca)

Clase	Qué hace	Relaciones clave	Mirá esto para validarla
VentanaPrincipal extiende JFrame	La ventana. Tiene el menú y la partida, y cambia entre ellos con un `CardLayout`.	◆ MenuPrincipal, ◆ PantallaJuego	Las pantallas no se conocen entre sí: se avisan con callbacks (`this::jugar`, `this::mostrarMenu`).
MenuPrincipal extiende JPanel	Menú con un botón por nivel y Salir, sobre el fondo de 1985.	◆ Cartel, usa Imagenes	Recibe `IntConsumer` y `Runnable`: no sabe quién lo contiene (bajo acoplamiento).
PantallaJuego extiende JPanel	La partida: HUD arriba, tablero al centro. Traduce teclas en movimientos y muestra pausa y resultado.	→ Juego, ◆ PanelHUD, ◆ PanelTablero, crea Cartel	Es la clase con más tareas (armar, teclado, carteles). Ver SRP en la sección 11.
PanelHUD extiende JPanel	Indicadores de nivel, movimientos y empujes, y botones Deshacer, Reiniciar y Pausa.	implementa ObservadorJuego, usa Juego e Imagenes	Los botones no toman el foco, así las flechas siempre mueven al jugador (`vista/PanelHUD.java:82`).
PanelTablero extiende JPanel	Dibuja el tablero celda por celda con las imágenes de la época, y el personaje mirando hacia donde se movió.	implementa ObservadorJuego, → Epoca, → Direccion, usa Imagenes	Los símbolos están en el mapa `CAPAS`: un elemento nuevo es una línea más, sin `switch` (`:35`).
Cartel extiende JPanel	Recuadro reutilizable con título, mensaje y botones.	lo usan el menú, la pausa y el resultado	Evita código duplicado: los tres carteles son la misma clase.
Imagenes «Singleton»	Caché de imágenes: cada PNG se lee del disco o del .jar una sola vez.	la usan Menu, HUD y Tablero; lee assets/	Ver sección 9 (Singleton).
Epoca enum · integración	Fondo, piso y pared de cada época (1955, 1985, 2015). Los niveles las van rotando.	la usa PanelTablero	Una época nueva es una constante más (OCP). `Math.floorMod` evita índices negativos.

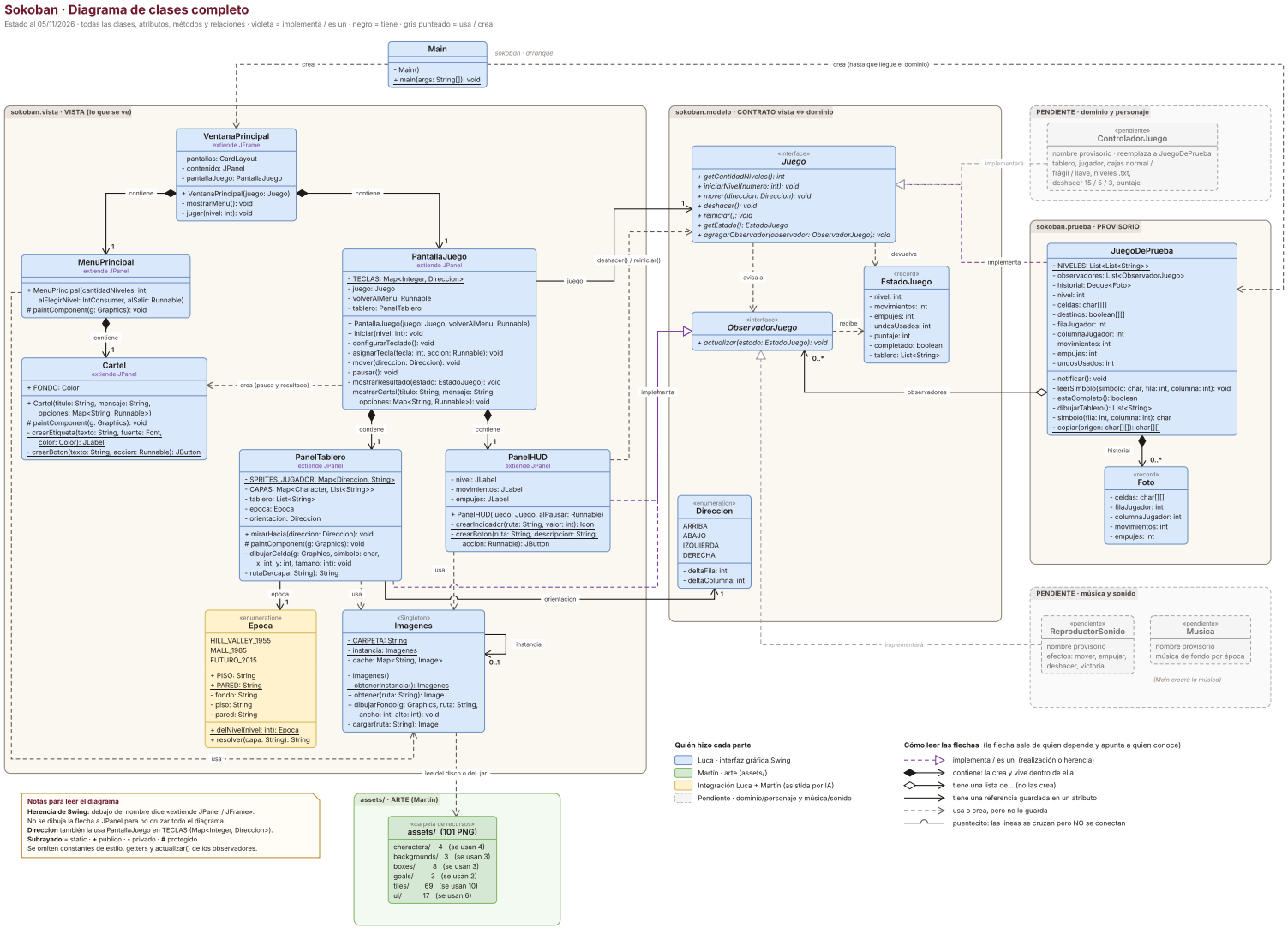
5.2 Contrato · `sokoban.modelo` (Luca)

Clase	Qué hace	Relaciones clave	Mirá esto para validarla
Juego interface	Todo lo que la vista le puede pedir al dominio: iniciar nivel, mover, deshacer, reiniciar, pedir el estado y registrar observadores.	Facade y sujeto del Observer	El comentario de `modelo/Juego.java:10` fija la regla: después de cada cambio hay que avisar a los observadores.
ObservadorJuego interface	Un solo método: `actualizar(EstadoJuego)`.	la implementan PanelHUD y PanelTablero	Un método solo: cumple ISP.
EstadoJuego record (DTO)	La foto del nivel: nivel, movimientos, empujes, deshacer usados, puntaje, si ganó y el tablero como líneas de texto.	lo devuelve Juego y lo recibe cada observador	Es inmutable: `record` + `List.copyOf` (`modelo/EstadoJuego.java:16`).
Direccion enum	ARRIBA, ABAJO, IZQUIERDA, DERECHA; cada una sabe cuánto se mueve en filas y columnas.	la usan PantallaJuego, PanelTablero y Juego	Evita «primitive obsession» (no se usan números o textos sueltos) y es Experto en información (GRASP).

5.3 Arranque y prueba (Luca)

Clase	Qué hace	Relaciones clave	Mirá esto para validarla
Main	Crea el juego y la ventana. Es la «raíz de composición»: el único lugar que conoce las clases concretas.	crea VentanaPrincipal y JuegoDePrueba	Usa `SwingUtilities.invokeLater`, como pide Swing.
JuegoDePrueba provisorio	Un Sokoban mínimo de 3 niveles (paredes, pisos, destinos y cajas normales) para probar la interfaz.	implementa Juego, ◇ ObservadorJuego, ◆ Foto	No cumple las reglas del deshacer (15 / 5 / 3) ni tiene cajas especiales: no hace falta, porque lo reemplaza el dominio.
Foto record interno	Copia del tablero y contadores para poder deshacer.	la guarda JuegoDePrueba en un historial	Se parece a un Memento (GoF) simplificado, pero no está planteado como patrón.

6 Diagrama de clases completo



docs/uml/2-diagrama-completo · es vectorial: hacé zoom en el PDF para leer los métodos. Los «puentecitos» marcan líneas que se cruzan sin conectarse.

7 Patrón Observer

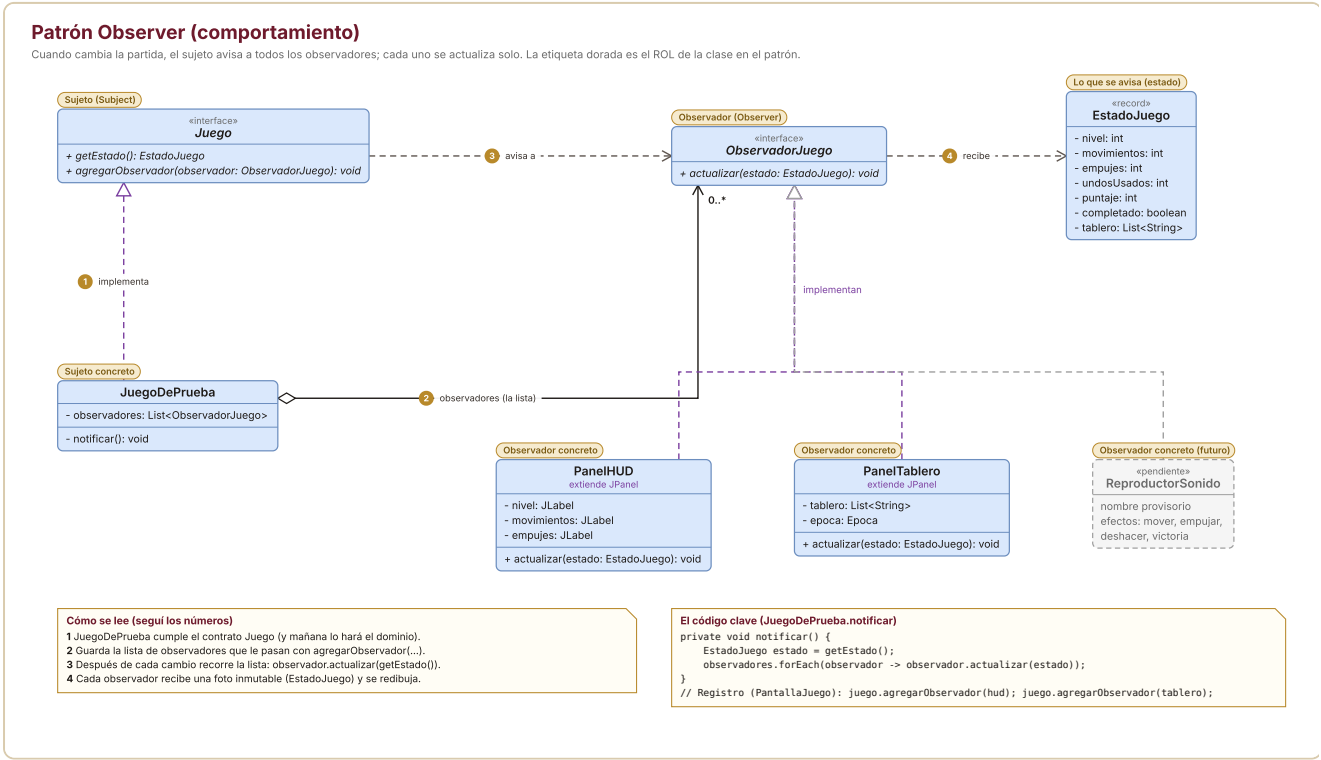
> *Sirve para que el HUD y el tablero se actualicen solos cada vez que cambia la partida, sin que el dominio sepa que existen.*

Definición

Observer es un patrón de comportamiento que define una dependencia de uno a muchos entre objetos: cuando un objeto (el sujeto) cambia de estado, todos sus dependientes (los observadores) son notificados y se actualizan automáticamente.

El problema que resuelve acá

Después de cada movimiento hay que redibujar el HUD y el tablero, y mañana también hacer sonar un efecto. Si el dominio llamara directamente a `PanelHUD` y `PanelTablero`, el dominio dependería de la vista (y de Swing). Con Observer el dominio solo conoce la interfaz `ObservadorJuego`.



docs/uml/3-patron-observer · seguí los números del 1 al 4.

Rol en el patrón (GoF)	Clase del proyecto	Dónde
Subject (sujeto)	`Juego` (`agregarObservador`)	`modelo/Juego.java:27`
ConcreteSubject	`JuegoDePrueba` (mañana el dominio)	`prueba/JuegoDePrueba.java:147`
Observer	`ObservadorJuego`	`modelo/ObservadorJuego.java:7`
ConcreteObserver	`PanelHUD` , `PanelTablero` (mañana el sonido)	`vista/PanelHUD.java:26` , `vista/PanelTablero.java:25`

Estado que viaja	`EstadoJuego` (modelo «push»)	`modelo/EstadoJuego.java:12`

prueba/JuegoDePrueba.java

```
142 @Override
143 public void agregarObservador(ObservadorJuego observador) {
144     observadores.add(observador);                  // guarda la INTERFAZ, no PanelHUD
145 }
146
147 private void notificar() {
148     EstadoJuego estado = getEstado();              // una sola foto para todos
149     observadores.forEach(observador -> observador.actualizar(estado));
150 }
```

7.1 Lista de chequeo

#	Pregunta	Dónde mirar	Estado
1	¿Hay una interfaz de observador con un método de aviso?	`ObservadorJuego.actualizar`	✓
2	¿El sujeto permite registrar observadores?	`Juego.agregarObservador`	✓
3	¿El sujeto guarda una lista del tipo interfaz (no de clases concretas)?	`List<ObservadorJuego>` en JuegoDePrueba	✓
4	¿Avisa después de **cada** cambio?	`notificar()` al final de `iniciarNivel`, `mover` y `deshacer` (`reiniciar` llama a `iniciarNivel`)	✓
5	¿El sujeto ignora quiénes son los observadores?	JuegoDePrueba no importa nada de `sokoban.vista`	✓
6	¿Se puede sumar un observador sin modificar el sujeto?	Ver el ejercicio resuelto (sección 13)	✓
7	¿Se puede dar de baja un observador?	No existe `quitarObservador`	⚠
8	¿Está garantizado que el dominio real va a notificar?	Solo lo dice un comentario en `Juego.java`	⚠

7.2 Veredicto

Bien aplicado: están los cuatro roles, el sujeto depende solo de la interfaz y sumar un observador no obliga a tocar nada. Para defenderlo, conviene saber estos matices:

- En el GoF el sujeto también tiene `detach` (dar de baja). Acá no hace falta porque el HUD y el tablero viven toda la ejecución; si algún día se crean y destruyen pantallas, habría que agregar `quitarObservador`.
- Cada implementación de `Juego` tiene que acordarse de llamar a `notificar()`: si el dominio se olvida, la pantalla no se actualiza. Una mejora posible es una clase abstracta (por ejemplo `JuegoObservable`) que ya traiga la lista y `notificar()`, para que el dominio herede eso hecho.
- Es un Observer «push»: el sujeto manda el estado completo. Como `EstadoJuego` es inmutable, ningún observador puede cambiarlo por accidente.

- `PantallaJuego` no es observador: después de mover consulta `getEstado()` para ver si ganó. Es válido (consulta directa), pero si el dominio terminara un nivel por otro camino (por ejemplo, una caja que resbala sola), el cartel no aparecería.

8 Patrón Facade

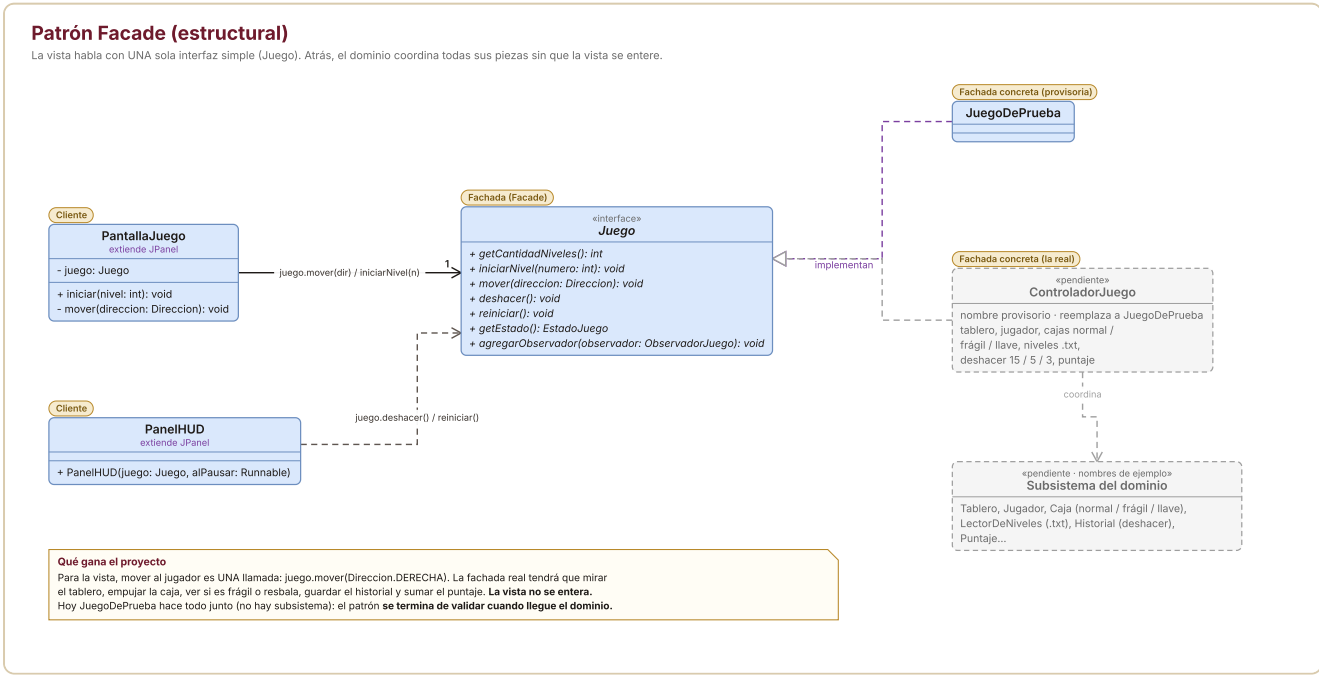
> *Sirve para que la vista hable con el dominio a través de una sola puerta simple, sin conocer las piezas que hay detrás.*

Definición

Facade es un patrón estructural que proporciona una interfaz unificada y simple a un conjunto de interfaces de un subsistema, para que el subsistema sea más fácil de usar.

El problema que resuelve acá

Mover al jugador en el dominio real implica mirar el tablero, empujar la caja, ver si es frágil o si resbala, abrir muros con la caja llave, guardar el historial y sumar el puntaje. La vista solo dice `juego.mover(Direccion.DERECHA)`.



docs/uml/4-patron-facade · el subsistema todavía no existe (gris).

Rol en el patrón (GoF)	Clase del proyecto	Dónde
Client	`PantallaJuego`, `PanelHUD`, `VentanaPrincipal`	usan solo métodos de `Juego`
Facade	`Juego` (interfaz) + la clase que la implemente	`modelo/Juego.java:13`
Subsystem classes	Tablero, jugador, cajas, niveles, historial, puntaje	× pendiente (dominio)

8.1 Lista de chequeo

#	Pregunta	Dónde mirar	Estado
1	¿Hay un punto de entrada único y con pocas operaciones?	`Juego` : 7 métodos	✓
2	¿Los clientes usan solo la fachada?	La vista no importa nada del dominio	✓
3	¿Hay detrás un subsistema de varias clases?	Hoy `JuegoDePrueba` hace todo en una clase	× todavía
4	¿La fachada delega en el subsistema en vez de resolver todo?	Se valida con el dominio real	⚠ pendiente
5	¿La interfaz tiene una sola responsabilidad?	`agregarObservador` mezcla «fachada» y «sujeto»	⚠

8.2 Veredicto

- La puerta (la interfaz) está bien diseñada: es chica, clara y protege a la vista de los cambios del dominio (GRASP «Variaciones protegidas»).
- Hoy el patrón está a medias: una fachada sin subsistema detrás es solo una interfaz. Se completa cuando el dominio implemente `Juego` con una clase que coordine `Tablero`, `Jugador`, las cajas, etc. Avisale a quien hace el dominio que su clase principal tiene que **delegar**, no hacer todo.
- En el GoF la fachada suele ser una clase concreta. Acá es una interfaz más una clase que la implementa: se combina Facade con DIP, lo que permite tener `JuegoDePrueba` mientras tanto.
- Si te preguntan por ISP: `Juego` podría dividirse en `Juego` (acciones) y `JuegoObservable` (registrar observadores). Con 7 métodos no es grave, pero es bueno saberlo.

9 Patrón Singleton

> *Sirve para que el menú, el HUD y el tablero compartan una única caché de imágenes, y cada archivo se lea una sola vez.*

Definición

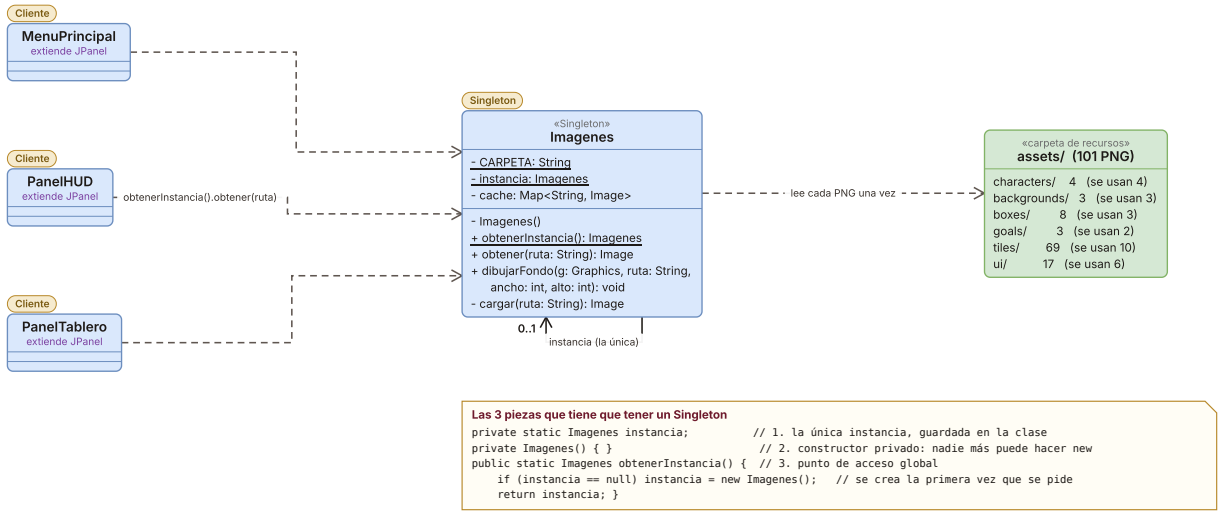
Singleton es un patrón creacional que garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.

El problema que resuelve acá

El tablero se redibuja en cada movimiento y usa decenas de imágenes. Leerlas del disco cada vez sería lento, y tener una caché por pantalla duplicaría memoria. Con una caché única, cada PNG se lee una vez.

Patrón Singleton (creacional)

Existe UNA sola caché de imágenes y todos la piden con Imagenes.obtenerInstancia(): cada PNG se lee del disco una vez.



Las 3 piezas que tiene que tener un Singleton

```
private static Imagenes instancia;          // 1. la única instancia, guardada en la clase
private Imagenes() { }                      // 2. constructor privado: nadie más puede hacer new
public static Imagenes obtenerInstancia() {  // 3. punto de acceso global
    if (instancia == null) instancia = new Imagenes();   // se crea la primera vez que se pide
    return instancia; }
```

docs/uml/5-patron-singleton

vista/Imagenes.java

```
18  public final class Imagenes {                    // final: nadie puede heredarla
19
20      private static final String CARPETA = "assets/";
21
22      private static Imagenes instancia;           // 1. la única instancia
23
24      private final Map<String, Image> cache = new HashMap<>();
25
26      private Imagenes() {                         // 2. constructor privado
27      }
28
29      public static Imagenes obtenerInstancia() {  // 3. acceso global
30          if (instancia == null) {                 //    se crea la primera vez (lazy)
31              instancia = new Imagenes();
32          }
33          return instancia;
34      }
```

9.1 Lista de chequeo

#	Pregunta	Dónde mirar	Estado
1	¿El constructor es privado?	`vista/Imagenes.java:26`	✓
2	¿Hay un atributo `static` con la única instancia?	`vista/Imagenes.java:22`	✓
3	¿Hay un método `static` de acceso que la crea si no existe?	`vista/Imagenes.java:29`	✓
4	¿La clase es `final`?	`vista/Imagenes.java:18`	✓
5	¿Realmente hace falta una sola instancia?	Sí: es una caché compartida	✓
6	¿Es seguro con varios hilos?	No está sincronizado	⚠ ver abajo

9.2 Veredicto

- Bien aplicado y bien justificado: tiene las tres piezas y un motivo real (caché compartida).
- No es seguro con varios hilos, pero en Swing todo el dibujo corre en un único hilo (el EDT) y `Main` arranca con `invokeLater`, así que hoy no hay riesgo. Si el sonido se carga en otro hilo, no debería usar esta clase.
- Críticas que te pueden hacer: es un estado global y dificulta los tests. La alternativa sería crear una sola `Imagenes` en `Main` y pasarla por constructor. Para una caché de solo lectura, el Singleton es defendible.
- En GRASP, `Imagenes` es una «Fabricación pura»: no representa nada del juego, existe para resolver un tema técnico.

10 MVC y el DTO `EstadoJuego`

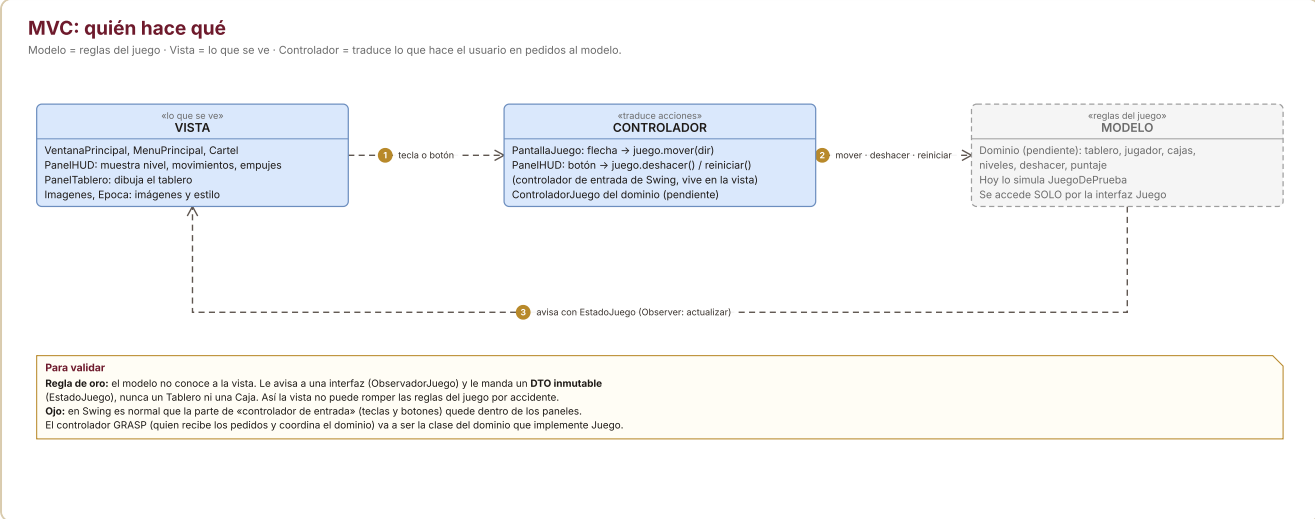
> *MVC no es un patrón GoF sino una forma de ordenar la aplicación: separa las reglas del juego (modelo) de lo que se ve (vista) y de cómo entran las acciones (controlador).*

Definición

MVC (Modelo-Vista-Controlador) divide la aplicación en el modelo, que tiene los datos y las reglas; la vista, que los muestra; y el controlador, que recibe las acciones del usuario y las traduce en operaciones sobre el modelo.

DTO

Un DTO (objeto de transferencia de datos) es un objeto simple, sin lógica, que solo lleva datos de una capa a otra. Acá es `EstadoJuego`: la vista recibe una foto del nivel y nunca un `Tablero` o una `Caja` del dominio.



docs/uml/6-mvc

10.1 Lista de chequeo

#	Pregunta	Dónde mirar	Estado
1	¿El modelo no conoce a la vista?	Sin imports de `sokoban.vista` en modelo ni prueba	✓
2	¿La vista recibe datos inmutables?	`record` + `List.copyOf`	✓
3	¿La vista modifica el modelo solo a través de operaciones?	Solo llama métodos de `Juego`	✓
4	¿Está claro quién es el controlador?	Repartido: ver abajo	⚠

- En Swing es normal que el «controlador de entrada» (teclas y botones) viva dentro de los paneles: `PantallaJuego` traduce las flechas y `PanelHUD` los botones.
- El «Controlador» de GRASP (la clase que recibe los pedidos del sistema y coordina el dominio) va a ser la clase del dominio que implemente `Juego`. Si te preguntan «¿dónde está el controlador?», explicá los dos niveles.
- El tablero viaja como texto con un carácter por celda, y no alcanza para dos cosas en la misma celda (caja llave sobre cerrojo, jugador sobre piso resbaladizo, caja frágil dañada). Hay que acordar con el dominio si se suman símbolos o se mandan dos capas (terreno y objetos).

11 SOLID y GRASP, con evidencia

> *Cada principio con el lugar exacto del código que lo muestra, y también dónde está flojo: en la defensa suma más reconocer un límite que exagerar.*

11.1 SOLID

Principio	Dónde se ve	Dónde está flojo	Estado
SRP · una sola responsabilidad	Cada clase hace una cosa: `Cartel` dibuja carteles, `Imagenes` carga imágenes, `Epoca` sabe el estilo de cada época.	`PantallaJuego` arma la partida, maneja el teclado y muestra los carteles. Se podría separar el teclado en otra clase.	⚠
OCP · abierto a extensión, cerrado a modificación	Nuevo observador, nuevo símbolo (`CAPAS`) o nueva época (`Epoca`) sin tocar la lógica existente.	Una tecla que no sea dirección obliga a editar `configurarTeclado()` .	✓
LSP · sustitución de Liskov	`JuegoDePrueba` y el dominio real son intercambiables detrás de `Juego` .	Se valida del todo cuando exista el dominio (tiene que notificar igual).	✓
ISP · interfaces chicas	`ObservadorJuego` tiene un solo método.	`Juego` mezcla acciones con el registro de observadores.	✓
DIP · depender de abstracciones	La vista depende de interfaces; solo `Main` conoce las clases concretas.	La vista sí depende de Swing, que es lo esperable.	✓

11.2 GRASP

Patrón GRASP	Dónde se ve en el proyecto	Estado
Experto en información	`Direccion` sabe cuánto se mueve; `Epoca` sabe qué piso y pared le tocan.	✓
Creador	`VentanaPrincipal` crea sus pantallas; `PantallaJuego` crea el HUD y el tablero que contiene.	✓
Controlador	Será la clase del dominio que implemente `Juego` .	× pendiente
Bajo acoplamiento	Las pantallas se avisan con `Runnable` / `IntConsumer` ; la vista solo conoce interfaces.	✓
Alta cohesión	Clases cortas con una sola tarea (ninguna pasa de ~120 líneas, salvo la provisoria).	✓
Polimorfismo	El juego llama `actualizar` sin saber si es el HUD, el tablero o el sonido.	✓
Fabricación pura	`Imagenes` : no es parte del juego, resuelve la carga de archivos.	✓
Indirección / Variaciones protegidas	`Juego` se interpone entre vista y dominio y protege a la vista de sus cambios.	✓

12 Lo que falta y los riesgos

> *Lo que todavía no está, en orden de importancia, para que nadie se lleve una sorpresa cerca de la entrega.*

Tema	Qué hay que hacer	Prioridad

Dominio y personaje	Implementar `Juego`: tablero, jugador, cajas normal / frágil / llave, piso resbaladizo, cerrojos y muros, niveles en .txt, puntaje.	alta
Reglas del deshacer	**«Se permite deshacer los últimos 15 movimientos realizados por el jugador.»** Cada uso vuelve 5 movimientos, con un máximo de tres usos consecutivos.	alta
Dos funcionalidades extra	**«Cada equipo debe, necesariamente, incorporar dos funcionalidades adicionales al juego que impliquen la aplicación de patrones de diseño.»** Todavía no están definidas.	alta
Celdas con dos cosas	El formato de un carácter por celda no alcanza (ver sección 10). Acordarlo antes de unir el dominio.	media
Sonido	Java reproduce WAV sin librerías extra; un observador más (ver sección 13). Romper una caja o abrir un cerrojo no se deduce del estado: hay que acordar un dato más.	media
StarUML	**«Diagrama de clases en StarUML»** Estos diagramas sirven para revisar, pero la entrega pide StarUML.	media
Licencias y uso de IA	Anotar el origen y la licencia de cada imagen y sonido, y declarar el uso de IA en el informe.	media

12.1 Ideas para las dos funcionalidades con patrones

Son sugerencias para discutir en el grupo; cada una nombra el patrón que aplicaría.

- Command + Memento para el deshacer. Cada movimiento es un objeto comando que se guarda en una pila limitada a 15, y el estado anterior se guarda como memento. Encaja con la regla de los 15 / 5 / 3.
- State para la partida. Estados «jugando», «en pausa» y «nivel completado», cada uno con lo que permite hacer.
- Strategy para el puntaje. Distintos criterios de puntaje intercambiables (por movimientos, por tiempo, por empujes).
- Factory Method para leer niveles. Crea la celda o la caja correcta a partir del carácter del archivo .txt.

13 Ejercicio resuelto: sumar el sonido sin tocar nada

> *Un ejemplo concreto de por qué Observer + OCP valen la pena: el sonido entra al juego como una clase nueva y una línea en Main.*

EJERCICIO RESUELTO Consigna: hacer que suene un efecto al moverse, al empujar, al deshacer y al ganar, sin modificar ninguna clase de la vista ni del dominio.

Resolución: el sonido es otro observador. Compara el estado nuevo con el anterior para saber qué pasó. Es un ejemplo para entender el patrón: la versión real la hace quien se ocupa del sonido.

ejemplo (no está en el repo)

```
public class ReproductorSonido implements ObservadorJuego {
    private EstadoJuego anterior;

    @Override
    public void actualizar(EstadoJuego estado) {
        if (anterior != null && estado.nivel() == anterior.nivel()) {
            if (estado.completado() && !anterior.completado()) {
                reproducir("victoria");
            } else if (estado.undosUsados() > anterior.undosUsados()) {
                reproducir("deshacer");
            } else if (estado.empujes() > anterior.empujes()) {
                reproducir("empujar");
            } else if (estado.movimientos() > anterior.movimientos()) {
                reproducir("paso");
            }
        }
        anterior = estado;
    }

    private void reproducir(String nombre) {
        // abrir assets/sonidos/<nombre>.wav con javax.sound.sampled.Clip
    }
}

// En Main, una línea:
juego.agregarObservador(new ReproductorSonido());
```

- No se modificó `Juego`, ni `JuegoDePrueba`, ni ninguna pantalla: eso es OCP.
- El juego la llama sin saber que es un sonido: eso es Polimorfismo (GRASP).
- Romper una caja frágil o abrir un cerrojo no se ve en `EstadoJuego`: para esos sonidos hay que sumar un dato al estado o un aviso aparte.

14 Autoevaluación

> *Tapá la columna de la derecha y respondé de memoria antes de mirar. Es la forma que más ayuda a llegar bien a la defensa.*

Pregunta	Respuesta
¿Por qué la vista puede funcionar antes de que exista el dominio?	Porque depende de la interfaz `Juego` y no de una clase concreta (DIP); mientras tanto la implementa `JuegoDePrueba`.
¿Qué cambia en el código cuando llegue el dominio?	Una línea en `Main` (`new JuegoDePrueba()` por la clase real) y se borra el paquete `prueba`.
En el Observer del proyecto, ¿quién es el sujeto y quiénes los observadores?	Sujeto: `Juego` (hoy `JuegoDePrueba`). Observadores: `PanelHUD` y `PanelTablero`.
¿Qué pasa si el dominio se olvida de llamar a `notificar()`?	El juego cambia pero la pantalla no se actualiza. La regla solo está escrita en un comentario.

¿Por qué `EstadoJuego` es un `record` con `List.copyOf`?	Para que sea inmutable: ningún observador puede modificar lo que reciben los demás.
¿Cuáles son las tres piezas de un Singleton y dónde están?	Atributo `static` con la instancia, constructor privado y método `static` de acceso: `Imagenes.java`, líneas 22, 26 y 29.
¿Por qué el Facade está «a medias» hoy?	Porque falta el subsistema: `JuegoDePrueba` hace todo en una sola clase.
¿Qué diferencia hay entre la flecha ◆ de `PantallaJuego` a `PanelHUD` y la punteada a `Cartel`?	◆ composición: lo crea y lo contiene toda la partida. Punteada: dependencia, lo crea en un método y no lo guarda.
¿Dónde está el controlador del MVC?	La entrada (teclas y botones) está en `PantallaJuego` y `PanelHUD`; el controlador GRASP será la clase del dominio que implemente `Juego`.
¿Cómo se suma el sonido sin romper OCP?	Con una clase que implemente `ObservadorJuego`, registrada en `Main` con `agregarObservador`.

Repasala en varias sesiones separadas (por ejemplo a los 1, 3, 7 y 14 días) en vez de todo junto la noche anterior.

15 Cómo mantener actualizados los diagramas

- Diagramas de clases: `python3 docs/uml/fuentes/diagramas.py` (las clases están en `clases.py` y la ubicación de cada caja y flecha en `diagramas.py`).
- Diagrama de secuencia: `docs/uml/fuentes/7-secuencia-mover.puml` (PlantUML).
- Esta guía: `node docs/guia/generar-pdf.js` a partir de `docs/guia/guia.html`.
- Cuando lleguen el dominio y el sonido, se reemplazan las cajas grises por las clases reales y se marcan con su autor.

Esta guía y los diagramas se hicieron con asistencia de IA a partir del código del repositorio, y conviene declararlo en el informe técnico. El equipo es responsable de revisarlos y entenderlos.